

PRACA DYPLOMOWA MAGISTERSKA

Analiza wydajności i efektywności wybranych algorytmów sortowania równoległego na wielordzeniowych procesorach

*A performance and efficiency analysis of selected
parallel sorting algorithms on multicore processors*

Krzysztof Pielot

Nr albumu: 133537

Kierunek: Informatyka

Forma studiów: stacjonarne

Poziom studiów: II

Promotor pracy:

dr inż. Bartosz Kowalczyk

Praca przyjęta dnia:

Podpis promotora:

Częstochowa, 2026

Niniejszym chciałbym serdecznie podziękować

...

*za bezcenne wsparcie udzielone mi
w trakcie trwania studiów.*

Spis treści

1	Wstęp	3
1.1	Cel pracy	4
1.2	Zakres pracy	5
2	Wprowadzenie teoretyczne	7
2.1	Wprowadzenie do sortowania równoległego	7
2.2	Charakterystyka procesorów wielordzeniowych	7
2.3	Wydajność i efektywność	7
2.4	Skalowalność algorytmów równoległych	7
3	Opis wybranych algorytmów sortowania równoległego	9
3.1	Parallel Quicksort	9
3.2	Parallel Merge Sort	10
3.3	Parallel Bucket Sort	11
3.4	Parallel Samplesort	13
4	Środowisko badawcze i implementacja	15
4.1	Opis środowiska	15
4.2	Wybór języka i bibliotek	16
4.3	Implementacja wybranych algorytmów	17
4.4	Charakterystyka danych testowych	23
5	Metodyka badań	27
5.1	Scenariusze testowe	27
5.2	Wariant sekwencyjny jako punkt odniesienia	28
5.3	Parametry pomiarów	29
5.4	Narzędzia do pomiaru czasu, CPU i pamięci	30
5.5	Przebieg eksperymentów	31
6	Analiza wyników badań	33
6.1	Porównanie czasu sortowania	33
6.2	Wpływ liczby rdzeni na wydajność	33
6.3	Analiza zużycia pamięci RAM i obciążenia CPU	33

6.4	Skalowalność algorytmów	33
6.5	Wpływ typu danych	33
6.6	Porównanie efektywności w zależności od charakterystyki danych wej- ściowych	34
7	Wnioski	35
	Podsumowanie	37
	Bibliografia	39
	Spis rysunków	41
	Spis tabel	42
	Listing	43
	Streszczenie	45
	Summary	47
	Słowa kluczowe	49
	Keywords	51

Rozdział 1

Wstęp

W ostatnim czasie można zaobserwować intensywny rozwój architektury procesorów wielordzeniowych. Producenci sprzętu zamiast dalszego zwiększania częstotliwości taktowania pojedynczego rdzenia decydują się na zwiększanie liczby rdzeni fizycznych oraz logicznych. Taka zmiana podejścia projektowania procesorów stawia przed twórcami oprogramowania nowe wyzwania. Klasyczne algorytmy sekwencyjne nie wykorzystują w pełni dostępnej mocy obliczeniowej współczesnych jednostek centralnych.

Szczególnie ważnym obszarem, w którym równoległość może przynieść wymierne korzyści, jest sortowanie danych. Sortowanie stanowi jeden z fundamentalnych problemów informatyki i znajduje szerokie zastosowanie m.in. w bazach danych, systemach plików, przetwarzaniu dużych zbiorów danych oraz uczeniu maszynowym. Wraz ze wzrostem objętości przetwarzanych informacji klasyczne algorytmy sortowania sekwencyjnego mogą coraz częściej stać się wąskim gardłem wydajnościowym.

Implementacja efektywnych algorytmów sortowania równoległego wiąże się z szeregiem wyzwań, takich jak odpowiedni podział danych, synchronizacja jednostek wykonawczych, zarządzanie dostępem do pamięci oraz ograniczanie narzutu wynikającego z realizacji obliczeń równoległych. W praktyce wiele istniejących rozwiązań albo pozostaje na poziomie teoretycznym, albo nie uwzględnia realnych ograniczeń sprzętowych i systemowych, takich jak koszt przełączania kontekstu, konflikty w dostępie do pamięci czy nierównomierne obciążenie rdzeni.

W związku z tym praktyczne zbadanie wydajności oraz efektywności wybranych algorytmów sortowania równoległego stanowi istotne i aktualne zagadnienie. W niniejszej pracy skoncentrowano się na implementacji i porównaniu czterech popularnych algorytmów: Parallel Quicksort, Parallel MergeSort, Parallel BucketSort oraz Parallel Samplesort. Algorytmy zostały zrealizowane w języku Python z wykorzystaniem biblioteki `multiprocessing`, a następnie poddane testom pod kątem czasu wykonania, wykorzystania pamięci RAM, obciążenia procesora, skalowalności oraz wpływu charakterystyki danych wejściowych.

1.1 Cel pracy

Głównym celem niniejszej pracy magisterskiej jest przeprowadzenie szczegółowej analizy wydajności i efektywności wybranych algorytmów sortowania równoległego na współczesnych wielordzeniowych procesorach. Praca skupia się na praktycznej implementacji oraz porównaniu czterech algorytmów: Parallel Quicksort, Parallel MergeSort, Parallel BucketSort oraz Parallel Samplesort.

Kluczowym celem jest zbadanie, w jaki sposób wybrane algorytmy zachowują się w realnych warunkach obliczeniowych, a nie jedynie w warunkach idealnych zakładanych przez modele teoretyczne. W tym celu algorytmy zostały zaimplementowane w języku Python z wykorzystaniem biblioteki `multiprocessing`, która umożliwia tworzenie procesów oraz współdzielenie pamięci. Każdy z algorytmów został uruchomiony zarówno w wariancie sekwencyjnym stanowiącym punkt odniesienia, jak i w wariancie równoległym, przy różnej liczbie wykorzystywanych rdzeni procesora.

Badanie obejmuje kilka kluczowych aspektów:

- ▶ pomiar czasu sortowania w zależności od rozmiaru zbioru danych oraz liczby użytych rdzeni,
- ▶ analizę zużycia pamięci operacyjnej (RAM) oraz obciążenia procesora (CPU) podczas wykonywania kodu,
- ▶ ocenę skalowalności, czyli zdolności algorytmów do efektywnego wykorzystania rosnącej liczby rdzeni,
- ▶ porównanie zachowania się algorytmów przy różnych charakterystykach danych wejściowych tj. dane losowe, częściowo posortowane oraz z duplikatami.

Dodatkowym celem pracy jest wyznaczenie i analiza klasycznych metryk równoległości, *speedup* (przyspieszenie) oraz *efficiency* (efektywność), a także identyfikacja głównych czynników ograniczających wydajność poszczególnych algorytmów (m.in. narzut związany z tworzeniem procesów, synchronizacją, komunikacją oraz zarządzaniem pamięcią współdzieloną).

W rezultacie pracy oczekuje się uzyskania praktycznych wniosków dotyczących tego, które z badanych algorytmów sortowania równoległego najlepiej sprawdzają się w środowisku wielordzeniowym przy określonych rozmiarach i typach danych, a także wskazania ich mocnych i słabych stron w kontekście rzeczywistego wykorzystania zasobów obliczeniowych.

1.2 Zakres pracy

Zakres pracy obejmuje:

- ▶ zebranie wiadomości z zakresu sortowania równoległego, architektury procesorów wielordzeniowych, metryk wydajności i efektywności oraz skalowalności algorytmów równoległych,
- ▶ porównanie wybranych algorytmów sortowania równoległego (Parallel Quicksort, Parallel MergeSort, Parallel BucketSort oraz Parallel Samplesort) pod kątem wydajności, skalowalności i zużycia zasobów obliczeniowych,
- ▶ opracowanie założeń projektowych dotyczących implementacji algorytmów w środowisku wielordzeniowym z wykorzystaniem biblioteki `multiprocessing` oraz pamięci współdzielonej,
- ▶ implementację wariantów sekwencyjnych i równoległych wybranych algorytmów sortowania w języku Python,
- ▶ implementację generatora danych testowych o różnych charakterystykach (dane losowe, częściowo posortowane, oraz dane z duplikatami),
- ▶ przetestowanie zaimplementowanych algorytmów na wielordzeniowych procesorach, obejmujące pomiar czasu wykonania, zużycia pamięci RAM, obciążenia CPU, wyznaczenie metryk *speedup* i *efficiency* oraz analizę wpływu charakterystyki danych wejściowych.
- ▶ prezentację wyników badań za pomocą tabel oraz wykresów.

————— Do uzupełnienia na końcu pracy —————

W rozdziale 2 przedstawiono podstawy teoretyczne sortowania równoległego, charakterystykę procesorów wielordzeniowych oraz metryki wykorzystywane do oceny wydajności i skalowalności. W rozdziale 3 opisano wybrane algorytmy sortowania równoległego. W rozdziale 4 scharakteryzowano środowisko badawcze, zastosowane technologie oraz sposób implementacji algorytmów. W rozdziale 5 przedstawiono metodykę badań, scenariusze testowe oraz narzędzia pomiarowe. W rozdziale 6 zaprezentowano i przeanalizowano wyniki przeprowadzonych eksperymentów. Pracę zamyka podsumowanie zawierające najważniejsze wnioski oraz możliwe kierunki dalszych badań.

Rozdział 2

Wprowadzenie teoretyczne

2.1 Wprowadzenie do sortowania równoległego

2.2 Charakterystyka procesorów wielordzeniowych

2.3 Wydajność i efektywność

2.4 Skalowalność algorytmów równoległych

Rozdział 3

Opis wybranych algorytmów sortowania równoległego

Dział poświęcony wybranym algorytmom zawierający opis, schemat działania, sposób dzielenia i łączenia, złożoność oraz potencjalne problemy.

3.1 Parallel Quicksort

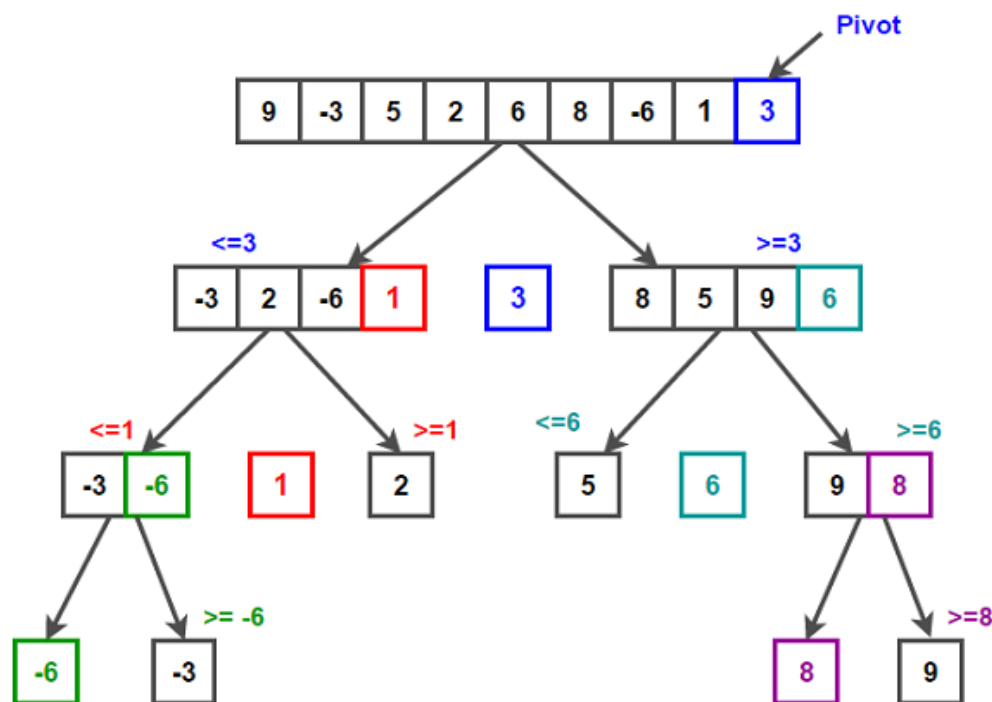
Quicksort należy do rodziny algorytmów opartych na strategii “dziel i zwyciężaj” (ang. *divide-and-conquer*) [1]. W kroku podziału wybierany jest element rozdzielający (ang. *pivot*), który dzieli wejściowy ciąg $A[p \dots r]$ na dwa podciągi. Po zakończeniu partycjonowania *pivot* trafia na pozycję $A[q]$, przy czym wszystkie elementy w podciągu $A[p \dots q - 1]$ są mniejsze lub równe $A[q]$, a wszystkie elementy w podciągu $A[q + 1 \dots r]$ są od niego większe. Następnie oba podciągi są rekurencyjnie sortowane aż do uzyskania podciągów o długości 1 [2].

Średnia złożoność obliczeniowa Quicksortu wynosi $O(n \log n)$, natomiast w przypadku pesymistycznym związanym z silnie niezrównoważonym podziałem może wzrosnąć do $O(n^2)$ [1].

W wersji równoległej algorytm wykorzystuje fakt, że po wykonaniu partycjonowania powstałe podciągi mogą być sortowane niezależnie od siebie. Dzięki temu możliwe jest równoległe przetwarzanie obu części tablicy przez osobne wątki. W praktycznych realizacjach poziom równoległości może być ograniczany poprzez ustalenie maksymalnej głębokości równoległej rekurencji, co umożliwia zmniejszenie nadmiernego narzutu związanego z tworzeniem i synchronizacją wątków. Po osiągnięciu założonej głębokości dalsze sortowanie odbywa się sekwencyjnie.

Jednym z głównych wyzwań wersji równoległej Quicksortu pozostaje nierównomierny podział danych, który występuje w sytuacji, gdy *pivot* jest źle dobrany. Problem ten, w połączeniu z narzutem związanym z tworzeniem i synchronizacją wątków, może istotnie ograniczać skalowalność algorytmu [3]. Algorytm sprawdza

się najlepiej przy danych zrównoważonych i dobrym wyborze pivotu, traci natomiast na efektywności m.in. gdy podziały stają się nierówne.



Rysunek 3.1: Schemat działania Quicksort.

3.2 Parallel Merge Sort

Merge Sort należy do algorytmów opartych na strategii “dziel i zwyciężaj” (ang. *divide-and-conquer*) [5]. Algorytm dzieli wejściowy ciąg na dwa podciągi o zbliżonej długości, następnie rekurencyjnie sortuje obie części, a na końcu scala je w jeden posortowany ciąg.

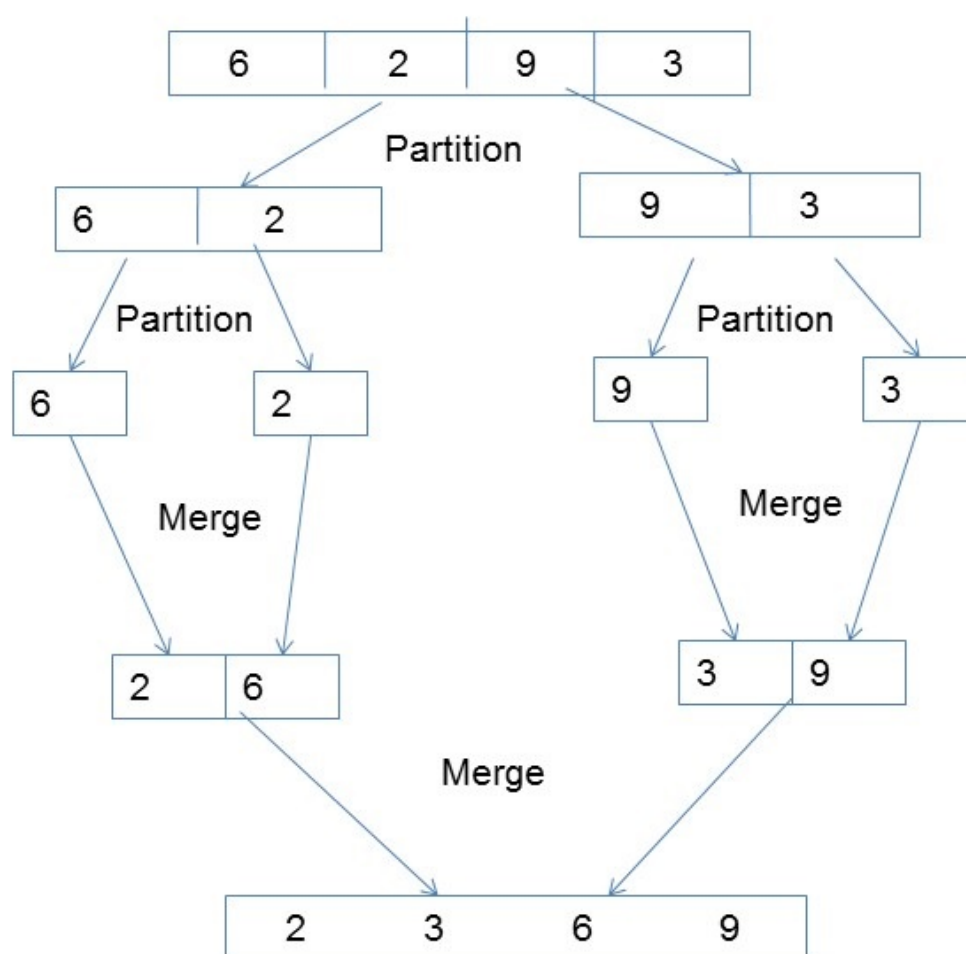
Operacja scalania stanowi kluczowy etap algorytmu i dla dwóch posortowanych podciągów o łącznej liczbie n elementów może zostać wykonana w czasie $O(n)$. Jeżeli zastosowana procedura scalania jest stabilna, cały algorytm Merge Sort również zachowuje stabilność [2].

Złożoność czasowa Merge Sort wynosi $O(n \log n)$. W przeciwieństwie do Quicksortu jej rząd nie zależy od początkowego uporządkowania danych, ponieważ algorytm w kolejnych krokach dzieli problem na dwa podproblemy o zbliżonych rozmiarach i wykonuje liniową operację scalania [5].

W wersji równoległej wykorzystuje się niezależność dwóch podproblemów powstałych po podziale danych. Rekurencyjne sortowanie obu części może dzięki temu odbywać się równolegle. Przed rozpoczęciem scalania konieczne jest jednak zakończenie sortowania obu podciągów, ponieważ etap scalania wykorzystuje wyniki obu wcześniejszych operacji [5]. Samo scalanie może być realizowane sekwen-

cyjnie lub równoległe.

Jednym z ograniczeń Parallel Merge Sort jest etap scalania. Wariant wykorzystujący równoległe sortowanie podciągów, lecz sekwencyjne scalanie, oferuje ograniczony poziom równoległości, ponieważ sekwencyjna operacja scalania staje się wąskim gardłem algorytmu [5]. Zastosowanie równoległego scalania pozwala zwiększyć dostępny poziom równoległości, jednak wiąże się z bardziej rozbudowanym sposobem łączenia posortowanych części. W praktycznych realizacjach dla odpowiednio małych podproblemów można również przejść do wykonania sekwencyjnego w celu ograniczenia narzutu związanego z równoległością [5].



Rysunek 3.2: Schemat działania Merge Sort.

3.3 Parallel Bucket Sort

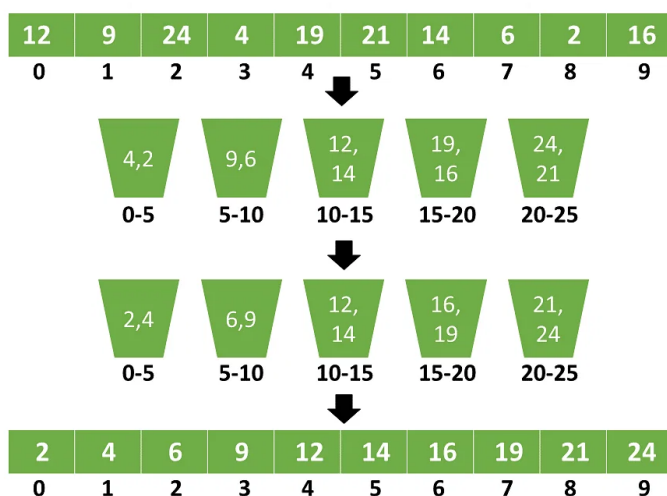
Bucket Sort jest algorytmem sortowania, którego działanie opiera się na podziale zakresu wartości danych wejściowych na określoną liczbę kubeków (ang. *buckets*). Każdy element wejściowy jest przypisywany do odpowiedniego kubka na podstawie swojej wartości, a następnie zawartość poszczególnych kubków jest sortowana niezależnie. Na końcu posortowane kubki są łączone w ustalonej kolejności,

tworząc wynikowy ciąg [5].

Przy założeniu odpowiedniego rozkładu danych wejściowych Bucket Sort może osiągać średnią złożoność czasową $O(n)$. W klasycznej analizie zakłada się równomierny i niezależny rozkład elementów, dzięki czemu można oczekiwać, że liczba elementów przypadających na poszczególne kubelki będzie zbliżona. Każdy kubek może być następnie sortowany za pomocą innego algorytmu, na przykład Insertion Sort, a ostateczny wynik powstaje przez konkatencję kubków w kolejności [5].

W wersji równoległej możliwe jest zrównoleglenie zarówno etapu rozmieszczania elementów w kubkach, jak i sortowania ich zawartości. Dane wejściowe mogą zostać podzielone na części, które są przetwarzane równoległe przez niezależne wątki. Po zakończeniu rozmieszczania elementów kubki mogą zostać podzielone na grupy, których sortowanie również odbywa się równoległe [6]. Po zakończeniu tej fazy posortowane kubki są łączone w odpowiedniej kolejności w jeden uporządkowany ciąg.

Efektywność Parallel Bucket Sort jest w dużym stopniu zależna od sposobu rozłożenia elementów pomiędzy kubkami. Przy równomiernym wypełnieniu kubków możliwe jest bardziej równomierne rozłożenie pracy pomiędzy jednostki wykonawcze. Jeżeli natomiast część kubków zawiera znacznie więcej elementów niż pozostałe, może prowadzić to do nierównomiernego obciążenia i ograniczenia uzyskiwanego przyspieszenia. Dodatkowym ograniczeniem jest narzut związany z realizacją obliczeń równoległych, w tym z synchronizacją oraz organizacją pracy pomiędzy jednostkami wykonawczymi. Dla małych zbiorów danych koszt ten może przewyższać korzyści wynikające ze zrównoleglenia, natomiast dla większych zbiorów możliwe jest uzyskanie lepszej wydajności [6].



Rysunek 3.3: Schemat działania Bucket Sort.

3.4 Parallel Samplesort

Rozdział 4

Środowisko badawcze i implementacja

4.1 Opis środowiska

Wszystkie eksperymenty zostały przeprowadzone na komputerze o następującej konfiguracji:

- ▶ **Procesor:** Intel(R) Core(TM) i9-9980XE CPU @ 3.00 GHz
- ▶ **Rdzenie fizyczne:** 4
- ▶ **Rdzenie logiczne:** 8
- ▶ **Taktowanie bazowe:** 2401 MHz
- ▶ **Pamięć operacyjna RAM:** 15,85 GB
- ▶ **System operacyjny:** Windows 10
- ▶ **Architektura:** AMD64 (x86-64)
- ▶ **Interpreter Python:** 3.11.7

Środowisko to umożliwia badanie zachowania algorytmów sortowania równoległego przy ograniczonej, ale realnej liczbie rdzeni. Obecność technologii Hyper-Threading (8 procesorów logicznych przy 4 rdzeniach fizycznych) pozwala dodatkowo ocenić wpływ logicznych jednostek wykonawczych na wydajność algorytmów.

Dane dotyczące sprzętu i oprogramowania były automatycznie zebrane i zapisywane w bazie danych SQLite przed przeprowadzaniem eksperymentów.

4.2 Wybór języka i bibliotek

Do implementacji algorytmów sortowania równoległego, genrowania danych, przeprowadzenia eksperymentów oraz wizualizacji wyników wybrano język programowania Python w wersji 3.11.7 [11]. Decyzja ta wynikała z kilku powodów. Python jest jednym z najpopularniejszych języków, charakteryzuje się przejrzystą i elegancką składnią, co ułatwia tworzenie, analizę i utrzymanie kodu. Posiada bogatą bibliotekę standardową oraz rozbudowany zbiór dostępnych bibliotek, pakietów i narzędzi programistycznych rozszerzających możliwości języka, które znacząco przyspieszają rozwój oprogramowania. Dodatkowo język ten jest w pełni przenośny między systemami operacyjnymi tj. Windows, Linux, macOS, co ma znaczenie przy planowaniu ewentualnych dalszych testów na innych platformach.

Niezwyczajnie istotną rolę w projekcie odgrywa biblioteka `multiprocessing` [12], stanowiąca część standardowej biblioteki Pythona. Pakiet ten przy użyciu interfejsu programistycznego podobnego do pythonowego modułu `threading` umożliwia tworzenie procesów oraz realizację obliczeń równoległych z pominięciem ograniczeń wynikających z Global Interpreter Lock (GIL). Dzięki temu możliwe jest efektywne wykorzystanie wielu rdzeni procesora. W ramach tej biblioteki wykorzystano również moduł `multiprocessing.shared_memory` [13], który pozwala na bezpośredni dostęp wielu procesów do wspólnego obszaru pamięci. Mechanizm ten eliminuje konieczność serializacji oraz przesyłania dużych struktur danych między procesami, ograniczając związany z tym narzut, co ma szczególne znaczenie podczas sortowania dużych zbiorów danych.

Biblioteka `NumPy` [14] obsługująca dane numeryczne i szybkie operacje na tablicach, została wykorzystana w optymalizacji algorytmów Parallel Bucket Sort oraz Parallel Samplesort. Posłużyła w nich do efektywnego rozdzielania elementów do kubeków, co znacznie przyspieszyło etap dystrybucji danych. Ponadto `NumPy` wykorzystano w module analizy złożoności obliczeniowej oraz przy generowaniu wykresów i rankingów.

Wyniki eksperymentów były przechowywane i przetwarzane z użyciem biblioteki `pandas` [15], która oferuje wygodne struktury danych (`DataFrame`) oraz narzędzia do filtrowania, agregacji i analizy tabelarycznej. Do generowania wykresów i wizualizacji wyników wykorzystano bibliotekę `matplotlib` [16].

Do monitorowania zużycia zasobów systemowych wykorzystano bibliotekę `psutil` [17] oraz narzędzia `py-cpuinfo` i `py-spy` [19].

Całość kodu źródłowego została opracowana w środowisku programistycznym JetBrains PyCharm [20], które zapewnia wsparcie dla języka Python, debugowanie oraz wygodne zarządzanie projektem.

Do przechowywania wygenerowanych danych testowych oraz wyników eksperymentów wykorzystano bazę danych SQLite [21]. Jest to wbudowany silnik bazo-

danowy, który nie wymaga osobnego serwera ponieważ odczytuje i zapisuje dane bezpośrednio do plików dyskowych. Rozwiązanie to upraszcza zarządzanie danymi i zapewnia łatwą przenośność między środowiskami.

Wybór powyższego zestawu technologii umożliwia na spójną implementację algorytmów, precyzyjny pomiar ich charakterystyk wydajnościowych oraz wygodną analizę i prezentację wyników badań.

4.3 Implementacja wybranych algorytmów

Wszystkie algorytmy zaimplementowano w języku Python z wykorzystaniem mechanizmu wieloprosesowego oraz pamięci współdzielonej. Każdy algorytm występuje w dwóch wariantach: sekwencyjnym, który stanowi punkt odniesienia oraz równoległym. Wspólną cechą implementacji równoległych jest przechowywanie sortowanych danych w bloku pamięci współdzielonej, mapowanym na tablicę typów `ctypes` (`c_longlong` dla liczb całkowitych oraz `c_double` dla liczb zmiennoprzecinkowych). Dzięki temu procesy potomne mogą odczytywać i modyfikować te same dane bez konieczności ich kopiowania oraz przesyłania pomiędzy procesami.

Ogólna architektura rozwiązania

Przy algorytmach opartych na strategii “dziel i zwyciężaj” czyli Parallel Quicksort oraz Parallel Merge Sort, zastosowano rekurencyjne tworzenie procesów z ograniczeniem głębokości równoległości za pomocą parametru `max_depth`. Dodatkowo wprowadzono progi minimalnego rozmiaru podproblemu zawarte w słowniku `PARALLEL_CUTOFF`, poniżej których dalsze tworzenie procesów jest nieopłacalne i następuje przejście do sortowania sekwencyjnego. Takie rozwiązanie ogranicza narzut związany z tworzeniem zbyt dużej liczby procesów przy małych fragmentach danych.

Dla algorytmów opartych na kubekach czyli Parallel Bucket Sort oraz Parallel Samplesort, zastosowano model z góry określonej liczby procesów `process_count`. Dane są najpierw rozdzielane do kubków, a następnie grupy kubków są przydzielane poszczególnym procesom. Również tutaj wprowadzono progi minimalnego rozmiaru zawarte w słownikach `PARALLEL_SIZE_CUTOFF` oraz `GROUP_SIZE_CUTOFF`, poniżej których algorytm przełącza się na wariant sekwencyjny.

Parallel Quicksort

Wariant sekwencyjny realizuje klasyczny Quicksort z wyborem pivota jako elementu środkowego.

```
1 def partition(arr, low, high):  
2     pivot = arr[(low + high) // 2]
```

```

3
4     i = low
5     j = high
6
7     while i <= j:
8         while arr[i] < pivot:
9             i += 1
10        while arr[j] > pivot:
11            j -= 1
12        if i <= j:
13            arr[i], arr[j] = arr[j], arr[i]
14            i += 1
15            j -= 1
16
17    return i

```

Listing 4.1: Funkcja partycjonowania w Quicksort

W wersji równoległej po wykonaniu partycji lewa i prawa część tablicy mogą być sortowane niezależnie przez osobne procesy. Każdy proces dołącza się do istniejącego bloku pamięci współdzielonej, sortuje przypisany dla siebie zakres, a następnie kończy działanie. Gdy rozmiar zakresu spada poniżej ustalonego progu lub osiągnięta zostaje maksymalna głębokość równoległości, dalsze sortowanie odbywa się sekwencyjnie.

```

1  if size <= min_size or depth >= max_depth:
2      sort_in_place_on_shared(arr, low, high)
3      return
4
5  local = arr[low:high + 1]
6  local_index = partition(local, 0, size - 1)
7  arr[low:high + 1] = local
8  index = low + local_index
9
10 left = (low, index - 1)
11 right = (index, high)
12
13 processes = []
14
15 for part in [left, right]:
16     p_low, p_high = part
17     if p_low >= p_high:
18         continue
19
20     part_size = p_high - p_low + 1
21     if part_size > min_size:
22         p = mp.Process(
23             target=parallel_quicksort_worker,
24             args=(shm_name, length, dtype, p_low, p_high,

```

```

25         depth + 1, max_depth, min_size)
26     )
27     p.start()
28     processes.append(p)
29     else:
30         sort_in_place_on_shared(arr, p_low, p_high)
31
32 for p in processes:
33     p.join()

```

Listing 4.2: Tworzenie procesów w Parallel Quicksort

Parallel Merge Sort

Wariant sekwencyjny realizuje klasyczny Merge Sort z rekurencyjnym podziałem tablicy na połowy oraz scalaniem posortowanych części.

```

1 def merge(arr, left, mid, right):
2     temp = [None] * (right - left + 1)
3
4     i = left
5     j = mid + 1
6     k = 0
7
8     while i <= mid and j <= right:
9         if arr[i] <= arr[j]:
10             temp[k] = arr[i]
11             i += 1
12         else:
13             temp[k] = arr[j]
14             j += 1
15
16         k += 1
17
18     while i <= mid:
19         temp[k] = arr[i]
20         i += 1
21         k += 1
22
23     while j <= right:
24         temp[k] = arr[j]
25         j += 1
26         k += 1
27
28     arr[left:right + 1] = temp

```

Listing 4.3: Funkcja scalania w Merge Sort

W wersji równoległej obie połowy mogą być sortowane jednocześnie przez osobne procesy. Po zakończeniu obu procesów następuje etap scalania, który w danej implementacji wykonywany jest sekwencyjnie. Głębokość równoległości oraz minimalny rozmiar zakresu ograniczają nadmierne tworzenie procesów.

```
1 if size <= min_size or depth >= max_depth:
2     sort_in_place_on_shared(arr, left, right)
3     return
4
5 mid = (left + right) // 2
6 processes = []
7
8 for part_left, part_right in [(left, mid), (mid + 1, right)]:
9     part_size = part_right - part_left + 1
10    if part_size > min_size:
11        p = mp.Process(
12            target=parallel_mergesort_worker,
13            args=(shm_name, length, dtype, part_left, part_right,
14                depth + 1, max_depth, min_size)
15        )
16        p.start()
17        processes.append(p)
18    else:
19        sort_in_place_on_shared(arr, part_left, part_right)
20
21 for process in processes:
22     process.join()
23
24 local = arr[left:right + 1]
25 merge(local, 0, mid - left, size - 1)
26 arr[left:right + 1] = local
```

Listing 4.4: Równoległe sortowanie połówek i sekwencyjne scalanie w Parallel Merge Sort

Parallel Bucket Sort

Wariant sekwencyjny rozdziela elementy do liczby kubełków wyznaczonej na podstawie rozmiaru danych, a następnie sortuje każdy kubełek z wykorzystaniem zaimplementowanego algorytmu Merge Sort. Na sam koniec łączy wyniki w jedną posortowaną tablicę.

```
1 def distribute_to_buckets(arr, bucket_count):
2     if len(arr) == 0:
3         return []
4
5     arr_np = np.asarray(arr)
6     min_value = arr_np.min()
```



```

7     max_value = arr_np.max()
8
9     if min_value == max_value:
10         buckets = [[] for _ in range(bucket_count)]
11         buckets[0] = list(arr)
12         return buckets
13
14     bucket_range = (max_value - min_value) / bucket_count
15     indices = np.minimum(
16         bucket_count - 1,
17         ((arr_np - min_value) / bucket_range).astype(np.int64)
18     )
19
20     order = np.argsort(indices, kind='stable')
21     sorted_indices = indices[order]
22     sorted_values = arr_np[order]
23
24     buckets = [[] for _ in range(bucket_count)]
25     boundaries = np.searchsorted(sorted_indices, np.arange(bucket_count +
26         1))
27     for i in range(bucket_count):
28         buckets[i] = sorted_values[boundaries[i]:boundaries[i + 1]].
29         tolist()
30
31     return buckets

```

Listing 4.5: Rozdzielanie elementów do kubeków w Bucket Sort

W wersji równoległej liczba utworzonych kubeków jest wyliczana na podstawie rozmiaru danych oraz liczby wykorzystywanych rdzeni. Po utworzeniu kubeków ich zawartość jest łączona w jedną tablicę, która umieszczana zostaje w pamięci współdzielonej, a zakresy poszczególnych kubeków są grupowane i przydzielane procesom. Procesy sortują przypisane im grupy kubeków niezależnie. Dla małych grup sortowanie wykonywane jest sekwencyjnie, co ogranicza narzut związany z tworzeniem procesów. Do efektywnego rozdzielania elementów wykorzystano operacje wektorowe biblioteki NumPy.

```

1 bucket_groups = split_bucket_ranges(bucket_ranges, process_count)
2 processes = []
3 sequential_groups = []
4
5 for group in bucket_groups:
6     if not group:
7         continue
8
9     group_size = group[-1][1] - group[0][0] + 1
10
11     if should_spawn_for_group(group_size, process_count):

```

```

12         process = mp.Process(
13             target=bucket_worker,
14             args=(shm.name, len(arr), dtype, group)
15         )
16         process.start()
17         processes.append(process)
18     else:
19         sequential_groups.append(group)
20
21 for group in sequential_groups:
22     bucket_worker_inline(arr, group)
23
24 for process in processes:
25     process.join()

```

Listing 4.6: Przydzielanie grup kubełków procesom w Parallel Bucket Sort

Parallel Samplesort

Wariant sekwencyjny działa w kilku etapach: podział danych na części, lokalne sortowanie, wybór próbek, wyznaczenie wartości rozdzielających, rozdzielanie elementów do kubełków według wyznaczonych wartości oraz ostateczne sortowanie kubełków.

```

1 def distribute_to_buckets(data, pivots):
2     if not pivots:
3         return [list(data)]
4
5     arr = np.asarray(data)
6     pivots_arr = np.asarray(pivots)
7
8     indices = np.searchsorted(pivots_arr, arr, side='right')
9
10    order = np.argsort(indices, kind='stable')
11    sorted_indices = indices[order]
12    sorted_values = arr[order]
13    boundaries = np.searchsorted(sorted_indices, np.arange(len(pivots) +
14        2))
15
16    buckets = []
17    for i in range(len(pivots) + 1):
18        buckets.append(sorted_values[boundaries[i]:boundaries[i + 1]].
19            tolist())
20
21    return buckets

```

Listing 4.7: Rozdzielanie elementów do kubełków na podstawie wartości rozdzielających w Samplesort

W wersji równoległej etapy te realizowane są z wykorzystaniem wielu procesów. Na początku procesy równoległe sortują lokalne fragmenty danych i zbierają próbki. Na podstawie próbek wyznaczone są wartości rodzielające, po czym dane elementy są rozdzielane do kubeków. Na końcu grupy kubeków są ponownie sortowane równoległe. W przypadku niewielkich grup danych stosowany jest wariant sekwencyjny, co ogranicza koszt związany z tworzeniem procesów. Do dystrybucji elementów wykorzystano operacje wektorowe biblioteki NumPy.

```
1 bucket_groups = split_bucket_ranges(bucket_ranges, process_count)
2 processes = []
3 sequential_groups = []
4 min_group_size = get_group_size_cutoff(process_count)
5
6 for group in bucket_groups:
7     if not group:
8         continue
9
10    group_size = group[-1][1] - group[0][0] + 1
11
12    if group_size > min_group_size:
13        process = mp.Process(
14            target=bucket_worker,
15            args=(shm.name, len(arr), dtype, group)
16        )
17        process.start()
18        processes.append(process)
19    else:
20        sequential_groups.append(group)
21
22 for group in sequential_groups:
23     sort_group(arr, group)
24
25 for process in processes:
26     process.join()
```

Listing 4.8: Równoległe sortowanie grup kubeków w Parallel Samplesort

4.4 Charakterystyka danych testowych

Do przeprowadzenia testów przygotowano zestawy danych o zróżnicowanej charakterystyce, tak aby możliwe było ocenienie zachowania algorytmów w różnych scenariuszach.

Typy i rozmiary danych

Testy przeprowadzono dla dwóch typów wartości:

- ▶ liczb całkowitych - `int`
- ▶ liczb zmiennoprzecinkowych - `float`

Dla każdego typu wygenerowano zbiory o następujących rozmiarach:

- ▶ 1 000 elementów
- ▶ 10 000 elementów
- ▶ 100 000 elementów
- ▶ 1 000 000 elementów

Wygenerowane wartości znajdują się w zakresie od 0 do 1 000 000.

Charakterystyka zbiorów

Przygotowano trzy główne kategorie danych:

- ▶ **Dane losowe** – elementy generowane w sposób całkowicie losowy z użyciem generatora liczb pseudolosowych.
- ▶ **Dane z duplikatami** – zbiory, w których około 40% elementów stanowią powtórzenia. Najpierw wygenerowano 60% unikalnych wartości zbioru o danym rozmiarze, a następnie na ich podstawie dodano brakującą część poprzez losowe powtórzenia.
- ▶ **Dane częściowo posortowane** – zbiory zawierające odpowiednio 20%, 40%, 60% oraz 80% elementów uporządkowanych. Posortowane fragmenty przeplatano fragmentami losowymi, co pozwoliło uzyskać zbiory o określonym stopniu częściowego uporządkowania.

```
1 def generate_part_sorted_values(table_name, count, scope, sorted_ratio):
2     sorted_count = int(count * sorted_ratio)
3     random_count = count - sorted_count
4     parts = 5
5
6     if "int" in table_name:
7         sorted_values = sorted(random.randint(0, scope) for _ in range(
8             sorted_count))
9         random_values = [random.randint(0, scope) for _ in range(
10             random_count)]
11     else:
```

```

10         sorted_values = sorted(random.uniform(0, scope) for _ in range(
            sorted_count))
11         random_values = [random.uniform(0, scope) for _ in range(
            random_count)]
12
13         chunk_size = sorted_count // parts
14         sorted_chunks = [sorted_values[i*chunk_size:(i+1)*chunk_size] for i
            in range(parts-1)]
15         sorted_chunks.append(sorted_values[(parts-1)*chunk_size:])
16
17         random_chunk_size = random_count // (parts + 1)
18         random_chunks = [random_values[i*random_chunk_size:(i+1)*
            random_chunk_size] for i in range(parts)]
19         random_chunks.append(random_values[parts*random_chunk_size:])
20
21         combined = []
22         for i in range(parts):
23             combined.extend(random_chunks[i])
24             combined.extend(sorted_chunks[i])
25         combined.extend(random_chunks[-1])
26
27         values = [(val, count) for val in combined]
28
29         return values

```

Listing 4.9: Generowanie danych częściowo posortowanych

Do testów przygotowano dwanaście wariantów zbiorów (2 typy danych X 6 charakterystyk).

Sposób przechowywania

Wszystkie dane testowe zapisano w bazie danych SQLite. Każdy wariant przechowywany był w osobnej tabeli o strukturze:

- ▶ `id` – unikalny identyfikator rekordu,
- ▶ `value` – wartość elementu `INTEGER` lub `REAL`
- ▶ `set_size` – rozmiar zbioru, do którego należy dany element, ograniczony do wartości 1000, 10 000, 100 000, 1000 000

Rozdział 5

Metodyka badań

5.1 Scenariusze testowe

Eksperymenty przeprowadzono według różnych konfiguracji parametrów, których celem jest zebranie informacji potrzebnych do przeprowadzenia analizy.

Zakres badanych konfiguracji

W badaniach uwzględniono cztery algorytmy:

- ▶ Quick Sort
- ▶ Merge Sort
- ▶ Bucket Sort
- ▶ Sample Sort

Dla każdego algorytmu wykonano pomiary jego wariantu sekwencyjnego oraz wersję równoległą. Wariant sekwencyjny stanowił punkt odniesienia i był zawsze uruchamiany na jednym rdzeniu.

Testy przeprowadzono dla następujących rozmiarów danych:

- ▶ 1 000 elementów
- ▶ 10 000 elementów
- ▶ 100 000 elementów
- ▶ 1000 000 elementów

Wykorzystano dwanaście wariantów zbiorów wejściowych, obejmujących:

- ▶ dane losowe
- ▶ dane z duplikatami

- ▶ dane częściowo posortowane (20%, 40%, 60% oraz 80% elementów uporządkowanych)

zarówno dla liczb całkowitych, jak i zmiennoprzecinkowych.

Konfiguracja równoległości

Testy równoległe wykonano dla dostępnych konfiguracji liczby rdzeni udostępnianych przez środowisko sprzętowe. W przypadku algorytmów Quick Sort oraz Merge Sort liczba procesów była zależna od parametru głębokości równoległości:

$$max_depth = \log_2(cores).$$

Dla Bucket Sort oraz Samplesort liczbę procesów ustawiano jako liczbę wykorzystanych rdzeni.

Organizacja eksperymentów

Każdy pomiar wykonywano wielokrotnie, a do analizy przyjmowano wartości uśrednione. Liczbę powtórzeń ustalono na 10/100 dla każdego wariantu eksperymentu, aby uzyskać możliwie dokładną wartość średnią z przeprowadzonych pomiarów. Przed właściwymi pomiarami wykonano dodatkowe uruchomienie, którego wynik nie był uwzględniany w obliczeniach. Miało ono na celu przygotowanie środowiska do właściwych testów, jak również zprofilowanie uruchomionego algorytmu, dzięki czemu narzut pracy profilera nie wpływał na wyniki.

Przeprowadzone testy pozwoliły uzyskać zróżnicowany zestaw wyników, które umożliwiają analizę wpływu rozmiaru danych, liczby wykorzystywanych rdzeni oraz charakterystyki zbioru wejściowego na czas sortowania, zużycie pamięci oraz efektywność zrównoleglenia.

5.2 Wariant sekwencyjny jako punkt odniesienia

Każdy z badanych algorytmów, oprócz wersji równoległej posiada również swój wariant sekwencyjny. Stanowił on punkt odniesienia dla przeprowadzanych po nim testów wersji równoległej, umożliwiając określenie, jak wariant równoległy wypada na tle odpowiadającego mu wariantu sekwencyjnego.

Uzyskane wyniki z wersji sekwencyjnej wykorzystywano następnie do wyznaczenia metryk efektywności zrównoleglenia:

- ▶ przyspieszenia (*speedup*)
- ▶ efektywności (*efficiency*)

Wariant sekwencyjny jest wykonywany na pojedynczym rdzeniu, z wykorzystaniem identycznych danych wejściowych oraz tych samych metod pomiaru czasu sortowania, zużycia pamięci RAM i obciążenia procesora co w odpowiadającym mu wariantcie równoległym. Jednakowe warunki umożliwiają skupienie analizy na rzeczywistych różnicach pomiędzy wariantami i uzyskanymi przez nich wynikami.

5.3 Parametry pomiarów

Podczas każdego eksperymentu zbierane są wyniki pozwalające na późniejszą analizę wydajności oraz efektywności algorytmów oraz wykorzystania zasobów sprzętowych.

Podstawowym parametrem jest czas wykonania algorytmu, mierzony z wysoką rozdzielczością za pomocą funkcji `time.perf_counter`. Dla serii powtórzeń zapisywano:

- ▶ czas średni
- ▶ medianę
- ▶ odchylenie standardowe

Dodatkowo monitorowano zużycie zasobów systemowych:

- ▶ średnie obciążenie procesora CPU w trakcie działania algorytmu
- ▶ średnie zużycie pamięci operacyjnej RAM
- ▶ maksymalne zużycie pamięci operacyjnej RAM

Na podstawie czasu sekwencyjnego T_1 oraz czasu równoległego T_p wyznaczano metryki efektywności zrównoleglenia:

- ▶ przyspieszenie (*speedup*):

$$S_p = \frac{T_1}{T_p},$$

- ▶ efektywność (*efficiency*):

$$E_p = \frac{S_p}{p} = \frac{T_1}{p \cdot T_p},$$

gdzie p oznacza liczbę wykorzystanych rdzeni.

Wartości *speedup* oraz *efficiency* dla wariantu sekwencyjnego są pomijane

Wszystkie uzyskane wyniki zapisywano w bazie danych SQLite, co umożliwiło ich bezpieczne przechowywanie oraz wykorzystanie w dalszych etapach badań.

5.4 Narzędzia do pomiaru czasu, CPU i pamięci

Do realizacji potrzebnych pomiarów wykorzystano zestaw narzędzi i bibliotek umożliwiających precyzyjne rejestrowanie czasu wykonania oraz monitorowanie wykorzystania zasobów systemowych w trakcie działania algorytmów.

Pomiar czasu

Zapewnia ona dostęp do zegara o wysokiej rozdzielczości, przeznaczonego do pomiaru krótkich przedziałów czasu. Czas wykonania wyznaczany jest jako różnica wartości licznika zarejestrowanych przed rozpoczęciem i po zakończeniu badanego fragmentu kodu. Dzięki wysokiej rozdzielczości funkcja dobrze nadaje się do przeprowadzania pomiarów wydajnościowych. W implementacji CPython wykorzystywany zegar jest monotoniczny, dzięki czemu jego wartość nie może cofać się w czasie.

Monitorowanie CPU i pamięci RAM

Zużycie procesora oraz pamięci operacyjnej monitorowano z wykorzystaniem biblioteki `psutil` [17]. Pomiary realizowano w osobnym wątku, który w ustalonych odstępach czasu zapisanych w `sample_interval` odczytywał informacje o wykorzystanych zasobach. Dla zbiorów danych o rozmiarze nieprzekraczającym 10 000 odstęp wynosił 0,01 sekundy, natomiast dla pozostałych zbiorów 0,05 sekundy. Podczas każdego pomiaru odczytano:

- ▶ łączne obciążenie CPU procesu głównego oraz procesów potomnych
- ▶ zużycie pamięci RAM na podstawie wartości RSS

Dzięki uwzględnieniu procesów potomnych możliwe było poprawne oszacowanie wykorzystywanych zasobów przez algorytmy równoległe.

Profilowanie wydajności

Do analizy szczegółowej wydajności wykorzystano dwa narzędzia:

- ▶ `cProfile` [18] – wbudowany profiler Pythona, umożliwiający pomiar liczby wywołań oraz czasu wykonania poszczególnych funkcji
- ▶ `py-spy` [19] – zewnętrzny profiler próbkujący umożliwiający analizę działania procesu głównego i podprocesów oraz generowanie typu *flamegraph*

Profilowanie nie wpływało na wartości raportowane w głównych wynikach, ponieważ pierwsze uruchomienie, na którym działały profilery było pomijane przy obliczaniu statystyk.

Organizacja pomiaru

Każdy pojedynczy przebieg algorytmu uruchamiano w osobnym procesie roboczym, co zapewniało izolację pomiaru oraz zapewniało zabezpieczenie testów w przypadku awarii lub zawieszeniu badanego algorytmu. Wprowadzono limit czasowy ustawiony na 15 minut, po przekroczeniu którego proces był przerywany, a dany test otrzymywał status `TIMEOUT`.

Zebrane próbki CPU i RAM agregowano do wartości średnich oraz maksymalnych, a następnie wraz z metrykami czasu, *speedup* i *efficiency* zapisano w bazie danych.

5.5 Przebieg eksperymentów

Opis jak zostały przeprowadzone testy. Przygotowanie środowisk testowych. Sposób zapisywania i archiwizacji wyników. Procedura uruchomienia algorytmu itp.

Dopytać czy ta sekcja w ogóle ma sens !!!!!!!!!!!!!

Rozdział 6

Analiza wyników badań

6.1 Porównanie czasu sortowania

Prezentacja wyników pomiarów czasu wykonania dla każdego z algorytmów (dla różnych zbiorów danych, rozmiarów i typów danych). Omówienie i analiza

6.2 Wpływ liczby rdzeni na wydajność

Prezentacja i omówienie wyników pod względem użytych liczby rdzeni

6.3 Analiza zużycia pamięci RAM i obciążenia CPU

Prezentacja i omówienie zużycia pamięci i obciążenia w tym zestawienie średniego i maksymalnego zużycia

6.4 Skalowalność algorytmów

Ocena, jak dobrze algorytmy skalują się wraz ze wzrostem liczby rdzeni i rozmiarem danych

6.5 Wpływ typu danych

Porównanie wyników dla tych samych algorytmów, ale różnych typów danych wejściowych

6.6 Porównanie efektywności w zależności od charakterystyki danych wejściowych

Omówienie wyników dla danych losowych itp. Wskazanie, który algorytm radził sobie lepiej przy danym rodzaju danych

Rozdział 7

Wnioski

Podsumowanie

Podsumowanie wyników, najważniejsze wnioski, ocena czy cele pracy zostały osiągnięte, ograniczenia przeprowadzonych badań oraz możliwe kierunki dalszego rozwoju i optymalizacji zaimplementowanych algorytmów.

Bibliografia

- [1] Cormen T.H., Leiserson C.E., Rivest R.L., Stein C., *Introduction to Algorithms*, 3rd Edition, The MIT Press, 2009.
- [2] Božidar D., Dobravec T., *Comparison of parallel sorting algorithms*, Technical report, Faculty of Computer and Information Science, University of Ljubljana, November 2015.
- [3] Maus A., *A full parallel Quicksort algorithm for multicore processors*, Department of Informatics, University of Oslo.
- [4] Goodrich M.T., Tamassia R., *Algorithm Design and Applications*, Wiley, 2015.
- [5] Cormen T.H., Leiserson C.E., Rivest R.L., Stein C., *Introduction to Algorithms*, 4th Edition, The MIT Press, 2022.
- [6] Hong H., *Parallel Bucket Sorting Algorithm*, Computer Science Department, San Jose State University.
- [7] Grama A., Gupta A., Karypis G., Kumar V., *Introduction to Parallel Computing*, 2nd Edition, Addison-Wesley, 2003.
- [8] JáJá J., *An Introduction to Parallel Algorithms*, Addison-Wesley, 1992.
- [9] Hennessy J.L., Patterson D.A., *Computer Architecture: A Quantitative Approach*, 4th Edition, Morgan Kaufmann.
- [10] Intel, *Hyper-Threading Technology*, <https://www.intel.com/content/www/us/en/gaming/resothreading.html>, stan na dzień: 08.08.2026.
- [11] Python Software Foundation, *The Python Tutorial*, <https://docs.python.org/3/tutorial/index.html>, stan na dzień: 11.08.2026.
- [12] Python Software Foundation, *multiprocessing — Process-based parallelism*, <https://docs.python.org/3/library/multiprocessing.html>, stan na dzień: 11.08.2026.
- [13]

- [14] NumPy Developers, *NumPy documentation*, <https://numpy.org/doc/stable/>, stan na dzień: 11.08.2026.
- [15] The pandas development team, *pandas documentation*, <https://pandas.pydata.org/docs/>, stan na dzień: 11.08.2026.
- [16] Matplotlib Development Team, *Matplotlib documentation*, <https://matplotlib.org/>, stan na dzień: 11.08.2026.
- [17] Rodola G., *psutil documentation*, <https://psutil.readthedocs.io/en/latest/>, stan na dzień: 11.08.2026.
- [18] Python Software Foundation, *The Python Profilers*, <https://docs.python.org/3/library/profile.html>, stan na dzień: 11.08.2026.
- [19] Fredrikson B., *py-spy: Sampling profiler for Python programs*, <https://github.com/benfred/py-spy>, stan na dzień: 11.08.2026.
- [20] JetBrains, *PyCharm – The Python IDE*, <https://www.jetbrains.com/pycharm/>, stan na dzień: 11.08.2026.
- [21] Hipp D.R., *About SQLite*, <https://sqlite.org/about.html>, stan na dzień: 11.08.2026.

Spis rysunków

3.1	Schemat działania Quicksort.	10
3.2	Schemat działania Merge Sort.	11
3.3	Schemat działania Bucket Sort.	12

Spis tabel

Spis listingów

4.1	Funkcja partycjonowania w Quicksort	17
4.2	Tworzenie procesów w Parallel Quicksort	18
4.3	Funkcja scalania w Merge Sort	19
4.4	Równoległe sortowanie połówek i sekwencyjne scalanie w Parallel Merge Sort	20
4.5	Rozdzielanie elementów do kubków w Bucket Sort	20
4.6	Przydzielanie grup kubków procesom w Parallel Bucket Sort	21
4.7	Rozdzielanie elementów do kubków na podstawie wartości rozdzielających w Samplesort	22
4.8	Równoległe sortowanie grup kubków w Parallel Samplesort	23
4.9	Generowanie danych częściowo posortowanych	24

Streszczenie

Summary

Słowa kluczowe

algorytmy sortowania;
sortowanie równoległe;
procesory wielordzeniowe;
sortowanie szybkie;
sortowanie przez scalanie;
sortowanie kubełkowe;
Samplesort;
programowanie równoległe;
wydajność;
wykorzystanie CPU;
wykorzystanie RAM;
przyśpieszenie;
efektywność;
skalowalność;
Python;
multiprocessing;
pamięć współdzielona

Keywords

sorting algorithms;
parallel sorting;
multicore processors;
Quicksort;
Merge Sort;
Bucket Sort;
Samplesort;
parallel programming;
performance;
CPU utilization;
RAM usage;
speedup;
efficiency;
scalability;
Python;
multiprocessing;
shared memory

Imię i nazwisko: Krzysztof Pielot

Nr albumu: 133537

Kierunek: Informatyka

Wydział: Informatyki i Sztucznej Inteligencji

Oświadczenie autora pracy dyplomowej*

Oświadczam pod rygorem odpowiedzialności karnej, że złożona przeze mnie praca dyplomowa pt. „*Analiza wydajności i efektywności wybranych algorytmów sortowania równoległego na wielordzeniowych procesorach*” jest moim samodzielnym opracowaniem i nie zawiera treści uzyskanych w sposób niezgodny z obowiązującymi przepisami.

Jednocześnie oświadczam, że moja praca (w całości ani we fragmentach) nie była wcześniej przedmiotem procedur związanych z uzyskaniem tytułu zawodowego w Politechnice Częstochowskiej.

Wyrażam/~~nie wyrażam~~** zgodę/zgody na nieodpłatne wykorzystanie przez Politechnikę Częstochowską całości lub fragmentów wyżej wymienionej pracy w publikacjach Politechniki Częstochowskiej.

.....
podpis studenta

*W przypadku zbiorowej pracy dyplomowej, dołącza się oświadczenia każdego ze współautorów pracy dyplomowej.

*Niepotrzebne skreślić.